

DECO: An Edge Collaborative Framework for Congestion Control in Mobile Uplinks

Heng Xu, Jialong Li, Danyang Zheng, and Letian Li

Abstract—Mobile wireless uplink services impose diverse and time-varying demands on throughput, latency, and stability. Congestion control must therefore align transport behavior with service-dependent quality-of-service (QoS) operating regions rather than optimize a single metric. Existing endpoint-centric algorithms, however, struggle to track these drifting regions under mobility-induced network variation, and learning-based designs often rely on isolated endpoint exploration, leading to slow trial-and-error adaptation. We present DECO, a service-oriented end-edge collaborative framework for mobile uplink traffic. DECO decouples objective expression from local execution through service profiles and an independent control plane, enabling edge-assisted policy generation, shared experience reuse, bootstrap initialization, and graceful fallback while keeping terminal-side execution lightweight. We primarily instantiate DECO as distributed CC, where an edge-side deep reinforcement learning (DRL) policy engine learns profile-aware control from shared experience and the Terminal performs lightweight local execution. We use sender-based CC selection as a secondary realization to show that the same interface can support terminals running conventional sender-side controllers. Evaluations over 28 real-world wireless traces show that DECO provides differentiated service modes. Its throughput-oriented mode retains about 90%–96% of BBR’s utilization while reducing 99.9th-percentile delay by 27%–52%. Its delay-sensitive mode operates in the Copa/Vegas low-delay region and achieves about 15% lower tail delay on pedestrian traces. DECO also improves multi-flow fairness convergence, remains robust under degraded control-service conditions, and scales efficiently at the edge.

Index Terms—Congestion control, edge computing, mobile computing, quality of service

I. INTRODUCTION

The rapid growth of real-time video upload, cloud gaming, vehicular telemetry, and other interactive applications [39], [22], [24], [19], [4] has made wireless uplink traffic central to mobile computing. Unlike downlink-heavy delivery, these services push data to the network while imposing diverse quality-of-service (QoS) requirements on throughput, queueing delay, tail delay, and stability. Here, congestion control (CC) directly shapes application-level user experience.

CC must keep each connection near a service-appropriate operating region in the multidimensional QoS space. Different applications prefer different regions: vehicular telemetry prioritizes low latency, whereas high-definition streaming

prioritizes sustained upload efficiency. The first challenge is therefore target selection. A controller must determine which operating region best matches the current service objective, rather than simply pursue one universally optimal point.

Even after a suitable region is identified, wireless mobility makes this target difficult to maintain. Channel fading, handovers, interference, and varying base-station load shift the throughput-delay tradeoff. Consequently, the suitable operating region drifts over time, as illustrated in Fig. 1b. The second challenge is therefore target tracking. The controller must continuously follow a moving service objective under time-varying wireless conditions, rather than converge once to a fixed operating point.

Existing endpoint-centric CC designs are insufficient for these two challenges. Classical and model-based schemes [16], [7], [37], [30], [8], [6] rely primarily on local transport signals such as RTT, loss, and delivery rate, and usually encode a fixed operating preference, such as high utilization or low delay. Learning-based CC [29], [10], [11], [2], [3], [35] improves adaptability by learning from observed performance, but it commonly preserves the same endpoint-local exploration paradigm. A single terminal only observes its own limited trajectory, so entering a new environment requires trial-and-error probing. This slow cold-start process can introduce queue buildup, throughput loss, or instability before convergence [28]. In highly dynamic networks, the environment may change again before the local learner stabilizes.

Together, these limitations reveal an architectural gap: mobile uplink CC should not be treated solely as endpoint-local logic. QoS-aware and service-oriented CC designs [22], [15], [31], [12] can expose application objectives or select suitable controllers, but many of them remain tied to local adaptation logic or specific infrastructure support. The network edge can provide complementary support by aggregating telemetry and control outcomes across terminals sharing similar mobile contexts, such as access conditions and service profiles. In the DECO architecture, an edge-side Service unit uses cross-terminal experience and stronger compute resources to generate policy support, initialize new connections, and refine long-horizon control behavior, while the Terminal preserves fast local measurement and execution.

To bridge this gap, we present DECO, a service-oriented end-edge collaborative framework for mobile uplink traffic. The core idea of DECO is to decouple *objective expression* from *local execution*. Through an independent control plane, mobile applications declare service profiles that express their performance preferences, while the Service unit provides policy support using shared experience and profile-aware objec-

Heng Xu and Jialong Li are with the Faculty of Computer Science and Artificial Intelligence, Shenzhen University of Advanced Technology, China (e-mail: xuheng@suat-sz.edu.cn; lijialong@suat-sz.edu.cn).

Danyang Zheng is with the School of Computing and Artificial Intelligence, Southwest Jiaotong University, China (e-mail: dzheng5@swjtu.edu.cn).

Letian Li is with the Department of Information Engineering, The Chinese University of Hong Kong, China (corresponding author, e-mail: letianli@cuhk.edu.hk).

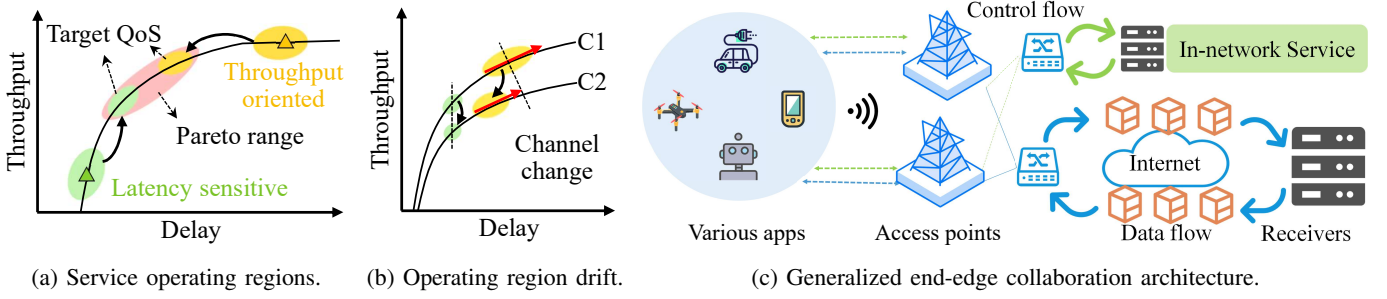


Fig. 1: Motivation for service-oriented end-edge collaborative congestion control.

tive modeling. The Terminal executes the returned policy locally and remains lightweight. By elevating congestion control into a reusable networked service capability, DECO enables edge-assisted policy generation, experience reuse, bootstrap initialization, and safe fallback under degraded control-service conditions.

DECO is not tied to any specific congestion-control algorithm and can support heterogeneous control realizations under the same service interface. In this paper, we develop two representative instances, namely Distributed CC and Sender-based CC selection. The former uses a Service-unit-assisted DRL policy engine with shared experience updates, while the latter performs profile-aware controller selection using historical outcomes and lightweight Terminal-side execution. This design emphasizes that DECO is an architectural framework rather than a congestion-control design tied to a single solver form.

We evaluate DECO over multiple real-world wireless traces covering stationary, pedestrian, and vehicular mobility scenarios. The results show that DECO provides differentiated, profile-aligned service modes within a unified framework. Across the three scenario types, the throughput-oriented mode retains about 90%–96% of BBR’s utilization while reducing P999 delay by 27%–52%, whereas the delay-sensitive mode remains in the Copa/Vegas region with comparable utilization and reduces tail delay by about 15% in pedestrian traces. We further validate robustness under control-service degradation, low runtime overhead, scalable edge execution, and faster adaptation through shared experience reuse.

The main contributions of this paper are summarized below.

- We reformulate mobile wireless uplink congestion control as the problem of defining, aligning with, and dynamically tracking service-dependent QoS operating regions under mobility-induced network dynamics, rather than optimizing a single metric in isolation.
- We propose DECO, a service-oriented end-edge collaborative congestion-control framework that decouples objective expression from local execution through service profiles, an independent control plane, and collaboration between the Terminal and the Service unit.
- We implement two representative DECO instances, namely Distributed CC and Sender-based CC selection, and validate their effectiveness over multiple real-world wireless traces, three mobility scenarios, and 12 baseline schemes.

The remainder of this paper is organized as follows. Section II presents the motivation and design rationale. Section III presents the DECO framework. Section IV develops the algorithmic instances. Section V evaluates the system. Section VI discusses related work. Section VII concludes the paper.

II. CHALLENGES AND MOTIVATION

In this section, we explain why wireless uplink congestion control should be viewed as service-dependent operating-region alignment rather than single-metric optimization (§II-A). We then derive the design rationale for a deployable collaborative solution and motivate DECO as the appropriate framework (§II-B).

A. Challenges of Wireless Uplink Congestion Control

For wireless uplink services, congestion control (CC) is not merely about preventing collapse or maximizing raw throughput [10], [11]. Its role is to keep a connection near a service-appropriate operating region over throughput, average delay, tail delay, and stability. As illustrated in Fig. 1a, different applications map to different favorable regions on the throughput-delay Pareto frontier. Delay-sensitive services prefer low-delay operation, whereas throughput-oriented services can tolerate more queueing in exchange for sustained utilization. The first challenge is therefore *target selection*: determining which operating region best matches the service objective. Under the conventional endpoint-local view, however, the sender observes only delayed and low-level feedback such as RTT, loss, and delivery rate, and thus has limited ability to infer the service-matching region on its own.

Even when a suitable region can be identified, the problem does not end there. Wireless uplink environments are highly dynamic: channel quality, access load, and competing traffic vary over time, which shifts the underlying throughput-delay tradeoff and causes the suitable region itself to drift, as shown in Fig. 1b. The second challenge is therefore *target tracking*: the controller must continuously follow a moving objective rather than converge once to a fixed point. When network conditions change, a Terminal that relies only on isolated local probing must repeatedly search for a new operating point. This trial-and-error process is often slow [18], [21], [38], creates unnecessary queueing and throughput oscillation during convergence, and can remain suboptimal if the environment changes again before stabilization.

These observations also clarify the problem addressed in this paper. The goal is *not* to identify one universally optimal congestion controller for all wireless uplink scenarios. Instead, the goal is to keep each connection as close as possible to the operating region that matches its service objective under dynamic wireless conditions. In other words, congestion control should be treated as a problem of defining, aligning, and continuously tracking service-dependent target regions, rather than optimizing a single metric or searching for one universally best endpoint algorithm.

B. Toward Deployable Collaborative Congestion Control

The above analysis suggests that endpoint-local congestion control is insufficient for wireless uplink services. Since target selection and target tracking both exceed what an isolated sender can reliably infer from delayed local feedback alone, some form of end-network collaboration is needed. The next question is how to realize such collaboration in a deployable way across heterogeneous real systems. Four design rationales follow directly from this requirement. First, the collaboration overhead must be low, so that the Terminal and the network-side support plane exchange only summarized and periodic information rather than heavy per-packet state or dense training updates. Second, the system should support rapid adaptation, so that a new or shifted connection can approach a suitable operating region without relying entirely on repeated online probing. Third, the behavior must remain stable under degraded control-service conditions, including delayed responses or temporary unavailability of the network-side support. Fourth, the design should remain compatible with heterogeneous endpoint implementations and scalable across many Terminals, access points, and service objectives.

These requirements expose gaps in existing works [22], [15], [31], [12]. Infrastructure-driven or network-assisted CC leverages network-side visibility, but many such designs are tightly coupled to specific access components or directly override endpoint behavior, which makes them harder to deploy across heterogeneous transport stacks. Distributed solution attempts to share experience across devices, but it usually assumes terminals with homogeneous learning capability and tolerates repeated model or gradient synchronization, which can be expensive for resource-constrained endpoints.

In both cases, the method may improve a particular congestion control realization, yet it does not by itself define a lightweight and reusable service profile for delivering experience to distributed endpoints. What is missing is the architectural layer that exposes intent, carries summarized telemetry, reuses historical experience, and delivers deployable policy support without taking away local execution autonomy.

Motivated by this gap, we elevate congestion control from an isolated end-host function to a general networked service. As depicted in Fig. 1c, DECO separates *objective expression* from *local execution*. Rather than dictating fine-grained sending actions or requiring every Terminal to participate in heavyweight distributed learning, it provides a deployable abstraction for policy support, experience reuse, bootstrap initialization, and safe fallback, while leaving local execution

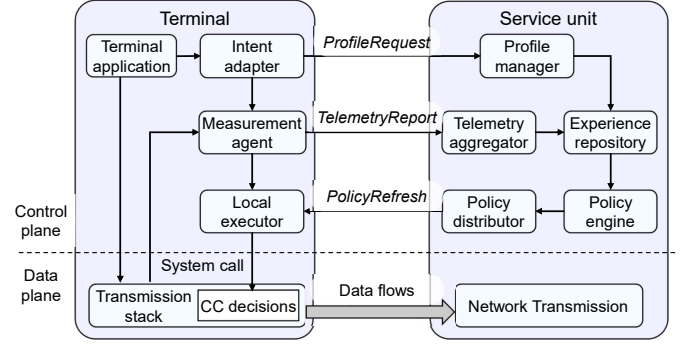


Fig. 2: Overview of DECO.

to the endpoint. In this sense, DECO is not a competing congestion-control algorithm. It is the framework that makes collaborative congestion control practical across heterogeneous wireless uplink environments.

III. SYSTEM FRAMEWORK

In this section, we define DECO by presenting its architecture (§III-A), service interface and objective modeling (§III-B), shared runtime update workflow (§III-C), and deployability mechanisms (§III-D).

A. Framework Overview

DECO realizes congestion control as a reusable end-edge service rather than a self-contained endpoint algorithm. We consider an end-network collaboration architecture with three logical entities: the *Terminal*, the access network, the *Service unit*. The Terminal sends application traffic, measures local states, and executes control actions. The Service unit provides congestion-control-related support but does not replace local execution.

Consistent with the end-network collaboration motivation introduced earlier, DECO organizes this collaboration through two logical planes, as shown in Fig. 2. The data plane carries the original application traffic and remains unchanged with respect to the normal transmission path. The control plane carries service declaration, summarized telemetry, policy generation, policy delivery, and runtime feedback. This separation turns the collaboration principle into a concrete framework structure, allowing the Service unit to provide network-side intelligence without taking over packet-level execution from the Terminal.

Within this structure, the Terminal and the Service unit play complementary roles. At the Terminal, intent adaptation, local measurement, and local execution connect application requirements to transport actions. At the Service unit, profile management, telemetry aggregation, experience reuse, policy generation, and policy distribution provide the shared support needed across connections and environments. This division keeps shared intelligence and long-horizon adaptation at the Service unit, while preserving fast local reaction and direct transport control at the Terminal.

Built on this division, DECO operates as a closed-loop collaboration between the Terminal and the Service unit. The

Terminal exposes intent, reports summarized observations, and executes the returned policy locally, while the Service unit provides shared support using environmental context and reusable experience. The intent are formalized in Section III-B, while the generic runtime update workflow shared by all DECO instances is detailed in Section III-C.

B. Service Interface and Objective Modeling

DECO exposes service requirements through *service profiles*. When a connection is established, the application does not request a specific congestion control algorithm. Instead, it declares a service type through a common interface. A service profile captures the desired tradeoff among throughput, latency, and stability, and thus serves as the objective-level input used by the Service unit to generate policy support.

In the basic configuration, DECO supports three representative profiles, namely delay-sensitive, balanced, and throughput-oriented. The delay-sensitive profile targets interactive services that prioritize low queueing delay and low tail latency. The throughput-oriented profile targets services that prioritize sustained upload efficiency. The balanced profile lies between the two. These profiles are not tied to fixed control parameters or particular algorithms. They are the minimal abstractions used in this paper to represent different operating tendencies.

DECO constrains the service interface but does not bind the internal solver. The Service unit may generate policy support through learning-based, rule-based, or hybrid mechanisms, as long as the generated support obeys the same service interface and serves the same profile-dependent objective. This solver-agnostic design is essential to the role of DECO as a framework rather than a single congestion-control algorithm.

Accordingly, a service profile is not merely a label for traffic classes, but the semantic entry point of objective modeling. For a declared profile ρ , DECO associates the connection with a profile-dependent target operating region in QoS space and evaluates policy quality by the deviation between realized performance and this target region. In this way, different solvers can produce different policies while still serving the same service semantics exposed by the interface.

To make this semantics explicit, we define a solver-independent shared service objective. The key idea is that different DECO instances should serve the same profile-dependent target operating region rather than optimize unrelated metrics. The objective is therefore defined independently of the internal solver and serves as the common semantic layer for all DECO realizations. We next formalize this semantics through a long-term performance description, a profile-dependent target operating region, and the corresponding deviation cost.

Let the service profile be denoted by ρ , the environment by e , and the control policy by π . The long-term statistical performance of a connection under policy π is written as:

$$\mathbf{y}_\pi(e) = (G_\pi(e), D_\pi(e), T_\pi(e), S_\pi(e)), \quad (1)$$

Here, $G_\pi(e)$, $D_\pi(e)$, $T_\pi(e)$, and $S_\pi(e)$ denote long-term effective throughput, average delay, tail-delay statistic, and stability, respectively.

Algorithm 1: Terminal-side Policy Execution Runtime

Input: Service profile ρ ; monitoring interval ΔT ; local fallback mode
Output: Local congestion control driven by profile-aware policy support

```

// 0. Bootstrap / Cold Start
1 Send ProfileRequest with  $\rho$  and Terminal capability to Service unit;
2 Receive bootstrap policy object through PolicyRefresh;
3 Initialize local executor with the received policy object;
4 while connection active do
    // 1. Policy Refresh
    5 if new PolicyRefresh is received and valid then
    6     Update the local policy object with the refreshed policy;
    // 2. Local Observation
    7 Observe local telemetry summary over the current monitoring interval  $\Delta T$ ;
    // 3. Local Policy Execution
    8 if the local policy object is valid and has not expired then
    9     Resolve local control decision  $a_t$  under the local policy object;
    10    Apply the corresponding local execution action;
    11 else
    12    Resolve local control decision  $a_t$  under the fallback mode;
    13    Execute the fallback mode locally;
    // 4. Outcome Construction
    14 Observe next-period telemetry summary;
    15 Derive profile-aware outcome summary from runtime behavior;
    // 5. Telemetry Upload
    16 Upload TelemetryReport carrying the current observation, action  $a_t$ , outcome summary, and next observation to Service unit;

```

For a given service profile ρ , the service objective is represented by an environment-dependent target operating region $\Omega_\rho(e)$ in the QoS space. In the present design, this region is made explicit through the dimensions directly enforced online, namely minimum throughput and delay-throughput tradeoff:

$$\Omega_\rho(e) = \left\{ (g, d) : g \geq g_\rho^{\min}, d \leq d_\rho^{\text{base}}(e) + k_\rho [g - g_\rho^{\min}]^+ \right\} \quad (2)$$

Here, $g_\rho^{\min}$ is the minimum acceptable throughput under ρ , $d_\rho^{\text{base}}(e)$ is the environment-dependent baseline delay budget, k_ρ characterizes the tolerated delay-throughput exchange, and $[x]^+ = \max(x, 0)$. A smaller k_ρ corresponds to a more delay-sensitive profile, whereas a larger k_ρ permits a more aggressive throughput-oriented tendency. Tail delay and stability remain part of long-term performance evaluation. In the present online realization, however, abnormal events are incorporated as penalty terms rather than hard boundaries in $\Omega_\rho(e)$.

Let

$$\mathbf{z}_t = (g_t, d_t, \bar{l}_t) \quad (3)$$

denote the online QoS state used in period t , where g_t , d_t , and $\bar{l}_t$ represent effective throughput, average one-way delay, and a

bounded abnormal-event indicator, respectively. Accordingly, the instantaneous deviation cost is written as

$$c_\rho(\mathbf{z}_t) = w_k^{(\rho)} \left[d_t - d_\rho^{\text{base}}(e_t) - k_\rho [g_t - g_\rho^{\min}]^+ \right]^+ + w_m^{(\rho)} [g_\rho^{\min} - g_t]^+ + w_l^{(\rho)} \bar{l}_t, \quad (4)$$

which penalizes two principal forms of region violation, namely excessive delay-throughput exchange and insufficient throughput, together with an abnormal event penalty used for robustness regularization.

The control objective is then defined as the minimization of the long-term deviation cost

$$J_\rho(\pi) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t c_\rho(\mathbf{z}_t) \right] \quad (5)$$

$\gamma \in (0, 1)$ is the discount factor. This formulation makes the semantics of DECO explicit: the goal is not the extreme value of any single metric, but the degree of matching between the realized QoS state and the target region associated with the declared service objective.

C. Runtime Update Workflow

Beyond the service interface and shared objective, DECO relies on a common runtime update workflow that is shared across its realizations. This workflow defines how the Terminal reports summarized experience, how the Service unit updates shared policy state, and how refreshed policy objects are fed back for local execution.

To preserve adaptation under time-varying wireless conditions, DECO refines policy support at the Service unit using newly generated sender-side experience. Rather than evolving each connection independently, DECO aggregates telemetry summaries and interaction outcomes from multiple Terminals into a shared experience pool, from which multiple worker threads asynchronously process the accumulated records and update the shared policy state for each service profile. This shared asynchronous design improves experience reuse across concurrent Terminals operating under similar environments or service profiles.

At runtime, each service profile is associated with a shared policy state at the Service unit. Worker threads process newly collected experience, refine the corresponding policy state, and periodically generate refreshed policy objects for Terminal-side execution. In this way, long-term policy evolution is moved from isolated Terminals to the Service unit, while the Terminal retains lightweight local measurement and execution. The generic procedures in Algorithms 1 and 2 summarize the reference Terminal-side execution loop and the reference Service-unit update loop, respectively; they are shared workflow descriptions rather than solver-specific training algorithms.

D. Bootstrap and Degradation Handling

DECO improves practical deployability through bootstrap support, experience reuse, and graceful degradation handling. When a new connection starts, the Service unit uses access context and similar historical cases in the Experience

Algorithm 2: Service-unit Policy Update and Refresh

Input: Service profiles; Experience Repository; shared policy states

Output: Bootstrap and refreshed policy objects for active Terminals

```

1 while Service unit running do
  // 1. Bootstrap / Cold Start
2  foreach new ProfileRequest received from a Terminal do
3    Resolve the requested service profile  $\rho$ ;
4    Retrieve historical experience for similar
      environments and Terminal capabilities;
5    Generate a profile-aware bootstrap policy object;
6    Return the bootstrap policy object through
      PolicyRefresh;
  // 2. Experience Aggregation
7  foreach received TelemetryReport do
8    Extract the reported observation, action, outcome
      summary, and next observation;
9    Insert the reported interaction into the Experience
      Repository of profile  $\rho$ ;
  // 3. Shared Policy Update
10 foreach service profile  $\rho$  with accumulated experience do
11   Invoke worker processing on the accumulated
     experience;
12   Refine the shared policy state of profile  $\rho$ ;
  // 4. Policy Refresh
13 foreach active Terminal do
14   if policy refresh condition holds then
15     Generate a refreshed policy object from the
       corresponding shared policy state;
16     Return the refreshed policy object through
       PolicyRefresh;

```

Repository to provide an initial policy that is better aligned with the requested service objective than uninformed cold-start probing. During runtime, newly reported telemetry and observed outcomes can be written back for future reuse, so later connections under similar conditions need not rediscover the same behavior from scratch.

DECO also keeps the control interaction coarse-grained in order to remain stable when the Service unit becomes slow or temporarily unavailable. Policy objects therefore carry validity periods and version identifiers. When fresh policies are delayed, the Terminal continues using the most recent valid policy. When the control plane becomes unavailable or a policy expires, the Terminal falls back to a local control mode. This design lets DECO benefit from network-side support without making the transport behavior fragile under degraded control-service conditions.

IV. INSTANCES

DECO can support terminals with different congestion-control capabilities. As illustrated in Fig. 3, a terminal may run a distributed AI-based CC controller, switch among mature sender-side CC algorithms, or apply a sender-side polish layer that adjusts the final congestion window of the underlying kernel CC algorithm. In this section, we focus on the first two cases, Distributed CC (Section IV-A) and Sender-based CC selection (Section IV-B), to show how different terminal-

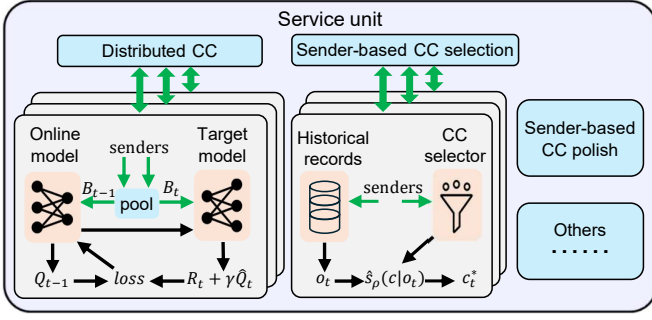


Fig. 3: Representative realization forms supported by DECO at the Service unit. This paper details two examples, while other forms such as sender-based CC polish are also compatible with the framework.

side CC capabilities can be mapped to the common DECO framework under the shared service semantics and runtime update workflow introduced in Sections III-B and III-C.

A. Primary Instance: Distributed CC

Our first instance introduces how DECO collaborates with distributed learning-based congestion control [32], where policy support maps observed network states to control actions through a shaped reward aligned with the service profile.

Transmission control is modeled as a partially observed closed-loop system. Let x_t denote the underlying network state, a_t the control action, and s_t the observation summary available at the Terminal. The system evolution is written as

$$x_{t+1} = f(x_t, a_t, \omega_t), \quad s_t = h(x_t), \quad (6)$$

where ω_t represents environmental disturbance. If the sending intensity u_t , queue length q_t , and service rate c_t are introduced explicitly, the queue dynamics can be written as

$$q_{t+1} = [q_t + u_t - c_t]^+. \quad (7)$$

Although the true network process is not strictly first-order Markov, practical control is performed over monitoring periods through summarized state observations, so next-period performance can be approximated as depending on the current summarized state and action.

Under service profile ρ , the control process is modeled as a Markov decision process

$$\mathcal{M}_\rho = (\mathcal{S}, \mathcal{A}, P, r_\rho, \gamma), \quad (8)$$

where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, P is the state transition law, r_ρ is the immediate reward, and $\gamma \in (0, 1)$ is the discount factor. In other words, the controller optimizes a shaped reward that serves as an online surrogate of $-c_\rho(\mathbf{z}_t)$.

The state is constructed from statistics that characterize the current operating point:

$$s_t = \phi(\text{rtt}_t, \text{min_rtt}_t, \text{thr}_t, \text{max_thr}_t, \text{cwnd}_t, \text{inflight}_t, \rho), \quad (9)$$

where $\phi(\cdot)$ denotes state generation and normalization. The state summarizes delay level, transmission efficiency, control intensity, queueing tendency, and the service profile.

The action adopts local incremental control:

$$a_t = \Delta \text{cwnd}_t \in \{-m, \dots, -2, -1, 0, 1, 2, \dots, m\}, \quad (10)$$

where m is the maximum adjustment magnitude. Positive actions increase sending intensity, negative actions decrease it, and zero keeps the current operating point unchanged. If pacing is supported, the same action can be mapped to pacing-rate adjustment.

Because the wireless uplink environment is difficult to model analytically, the transition law is treated in a model-free manner:

$$s_{t+1} \sim P(\cdot | s_t, a_t). \quad (11)$$

The approximate Markov property is established at the granularity of the monitoring period, where the state summarizes the dominant statistics governing next-step performance evolution.

To obtain a lightweight online objective consistent with the shared target operating region, we use a local incremental surrogate of region deviation. Let g_t and d_t denote the throughput and one-way delay metrics used for reward construction in period t . Their normalized increments are defined as:

$$\Delta \bar{g}_t = \frac{g_t - g_{t-1}}{g_{t-1} + \epsilon}, \quad \Delta \bar{d}_t = \frac{d_t - d_{t-1}}{d_{t-1} + \epsilon}, \quad (12)$$

where $\epsilon > 0$ is a small constant for numerical stability. Since directly computing the exact distance to $\Omega_\rho(e)$ online is difficult, throughput and delay increments are used to indicate whether the current action moves the connection toward or away from a more favorable operating point.

Given the profile-dependent tolerance k_ρ , we define the delay-throughput tradeoff violation as

$$\delta_t^{(\rho)} = [\Delta \bar{d}_t - k_\rho [\Delta \bar{g}_t]^+]^+, \quad (13)$$

which serves as an online surrogate for the violation of the profile-dependent delay-throughput envelope. Similarly, the minimum-throughput constraint in $\Omega_\rho(e)$ is represented by

$$\bar{m}_t = \left[\frac{g_\rho^{\min} - g_t}{g_\rho^{\min} + \epsilon} \right]^+, \quad (14)$$

which measures the normalized mismatch between the achieved throughput and the minimum throughput required by profile ρ . In addition, let $\bar{l}_t$ denote a bounded abnormal-event indicator.

We then define the profile-aware immediate reward as

$$r_t^{(\rho)} = w_g^{(\rho)} \Delta \bar{g}_t - w_d^{(\rho)} \Delta \bar{d}_t - w_k^{(\rho)} \delta_t^{(\rho)} - w_l^{(\rho)} \bar{l}_t - w_m^{(\rho)} \bar{m}_t. \quad (15)$$

The reward combines throughput improvement and delay reduction with penalties for delay-throughput envelope violation, abnormal events, and minimum-throughput mismatch, thereby providing a tractable online surrogate of the shared region-based objective.

The coefficients $w_g^{(\rho)}$, $w_d^{(\rho)}$, $w_k^{(\rho)}$, $w_l^{(\rho)}$, and $w_m^{(\rho)}$ are profile-dependent, so different services are realized through different parameterizations of the same reward structure. In general, throughput-oriented profiles place more weight on throughput gain, while delay-sensitive profiles place more weight on delay and violation penalties.

For distributed policy learning, we adopt a model-free action-value formulation. Let $Q_\rho(s, a; \theta)$ denote the Service-unit online action-value function under service profile ρ , and let $\hat{Q}_\rho(s, a; \theta^-)$ denote the corresponding target network. The training loss is

$$L(\theta) = \mathbb{E} \left[\left(r_t^{(\rho)} + \gamma \max_{a'} \hat{Q}_\rho(s_{t+1}, a'; \theta^-) - Q_\rho(s_t, a_t; \theta) \right)^2 \right]. \quad (16)$$

This objective drives the estimated action value toward the immediate profile-aware reward plus the optimal discounted return of the next state.

B. Secondary Instance: Sender-based CC selection

Our second instance is to select suitable congestion control for applications with their service objective [40], [38], [21], [26].

Let the candidate controller set be

$$\mathcal{C} = \{Vegas, BBR, Cubic\}. \quad (17)$$

In this case, the generic control action a_t is instantiated as a discrete controller choice c_t . Instead of applying a fine-grained congestion-window or pacing-rate update, the Service unit selects:

$$c_t \in \mathcal{C}. \quad (18)$$

At any time, the Terminal executes only the currently selected controller rather than probing multiple candidates concurrently. The expected effect of the other candidates is estimated by the Service unit from historical outcomes.

Under the shared runtime workflow, the Terminal continues to upload telemetry summaries through the control plane. Instead of making a new controller decision at every monitoring period, the Service unit aggregates every K consecutive monitoring periods into one selector window and performs controller selection at the granularity of this window. Let the telemetry summary of the t -th selector window be

$$o_t = \psi(\text{rtt}_t, \min_rtt_t, \text{thr}_t, \max_thr_t, \text{loss}_t, \text{inflight}_t, \rho), \quad (19)$$

where $\psi(\cdot)$ denotes the selector-summary construction function. The selector uses only short-window statistics that are already available in the control plane, such as delay inflation, throughput deficit, abnormal-event level, and queueing tendency.

To evaluate how well a candidate controller matches the shared service objective, we define a window-level QoS outcome vector

$$z_t = (\bar{g}_t, \bar{d}_t, \bar{\delta}_t, \bar{l}_t, \bar{m}_t), \quad (20)$$

where $\bar{g}_t$, $\bar{d}_t$, $\bar{\delta}_t$, $\bar{l}_t$, and $\bar{m}_t$ denote the normalized throughput term, delay term, delay-throughput tradeoff violation, abnormal-event indicator, and minimum-throughput mismatch measured over the selector window, respectively. Based on service profile ρ , the profile-aware selection score is

$$S_\rho(z_t) = w_g^{(\rho)} \bar{g}_t - w_d^{(\rho)} \bar{d}_t - w_k^{(\rho)} \bar{\delta}_t - w_l^{(\rho)} \bar{l}_t - w_m^{(\rho)} \bar{m}_t. \quad (21)$$

This score follows the shared service objective, so that delay-sensitive, balanced, and throughput-oriented profiles correspond to different parameterizations of the same objective.

The Experience Repository maintains historical records for different service profiles and candidate controllers. For controller $c \in \mathcal{C}$, let

$$\mathcal{E}_\rho(c) = \{(o_i, z_i)\}_{i=1}^{N_{\rho,c}} \quad (22)$$

denote the historical telemetry summaries and the corresponding observed QoS outcomes under profile ρ . Given the current telemetry summary o_t , the Service unit retrieves a small set of similar historical records for controller c , denoted by

$$\mathcal{N}_\rho(o_t, c) \subseteq \mathcal{E}_\rho(c), \quad (23)$$

and estimates the expected profile-aware score of controller c as

$$\hat{S}_\rho(c | o_t) = \frac{1}{|\mathcal{N}_\rho(o_t, c)|} \sum_{(o_i, z_i) \in \mathcal{N}_\rho(o_t, c)} S_\rho(z_i). \quad (24)$$

The controller selected for the current selector window is therefore

$$c_t^* = \arg \max_{c \in \mathcal{C}} \hat{S}_\rho(c | o_t). \quad (25)$$

That is, under the current service profile and measured state, DECO chooses the candidate controller whose historical outcomes are expected to be most aligned with the target operating region.

At initialization, the Service unit pre-fills the experience repository using offline measurements collected over representative wireless traces and candidate controllers. When similar historical records are still insufficient, a profile-dependent default controller is used as the bootstrap choice. During runtime, the Terminal continues to execute the currently selected controller and upload telemetry summaries, while the Service unit appends newly observed pairs (o_t, z_t) to the corresponding historical record set. In this way, the selector is initialized offline but continuously refined online through accumulated runtime outcomes.

To avoid frequent controller flapping, a switch is triggered only when the newly recommended controller provides a sufficiently large relative score improvement over the current one, namely

$$\frac{\hat{S}_\rho(c_t^* | o_t) - \hat{S}_\rho(c_t | o_t)}{|\hat{S}_\rho(c_t | o_t)| + \epsilon} > \beta_{\text{sw}}, \quad (26)$$

where $\beta_{\text{sw}} = 0.1$ is the switching threshold and $\epsilon > 0$ is a small constant for numerical stability.

Overall, the two instances share the same service semantics and runtime update workflow but differ in the internal solver and the granularity of the action space. The distributed CC instance learns a fine-grained local control policy, whereas the sender-based CC selection instance performs lightweight controller selection over a discrete candidate set.

V. EVALUATION

In this section, we evaluate DECO from three complementary perspectives. After describing the experimental setup (§V-A), we evaluate the two DECO instantiations, Distributed CC and Sender-based CC selection (§V-B1 and §V-B2). We then study multi-flow fairness convergence for Distributed CC

TABLE I: Parameter settings for DECO.

Symbol	Meaning	Value
m	Maximum action step	8
ΔT	Monitoring interval	80 ms
T_{req}	Refresh interval	5, 10, 20 s
$\mathbf{h}_Q$	Hidden-layer widths	[32, 64, 128, 64]
γ	Discount factor	0.95
η	Learning rate	10^{-3}
B	Mini-batch size	64
N_{worker}	Threads for each profile process	128
N_{replay}	Replay buffer size	20,000
K_{sync}	Target-network sync interval	200 iterations
ϵ	Numerical protection term	10^{-6}
k_ρ	Tradeoff threshold (s1/s2/s3)	1.3/1.5/2
$w_g^{(\rho)}$	Throughput-gain weight (s1/s2/s3)	0.6/0.8/1.0
$w_d^{(\rho)}$	Delay-cost weight (s1/s2/s3)	0.8/0.5/0.15
$w_k^{(\rho)}$	Tradeoff-violation weight (s1/s2/s3)	1.0/0.6/0.1
$w_l^{(\rho)}$	Penalty weight (s1/s2/s3)	0/0/0
$w_m^{(\rho)}$	Min-throughput weight (s1/s2/s3)	1.0/0/0

(§V-B3). Finally, we present a framework-oriented evaluation (§V-C1, §V-C2, and §V-C3), covering control-service degradation, overhead, scalability, experience reuse, and coexistence with legacy congestion control.

A. Setup

Hardware and Emulation Platform. Our experimental testbed consists of multiple sender-side hosts and an edge-cloud server. On the sender side, Mahimahi [23] emulation is distributed across multiple physical hosts, each running one or more pipelines configured with real-world wireless traces. A representative sender host is a laptop equipped with a 16-core, 32-thread CPU running at 5.4 GHz and Linux kernel 5.4.0-150. The edge-cloud service is deployed on a server equipped with an AMD EPYC 7B13 64-Core Processor, providing 64 CPU cores and 128 hardware threads. As a trace-driven network emulator, Mahimahi enables us to reproduce realistic wireless network dynamics from collected traces. Under this setup, concurrent senders can either compete with each other over different emulated links or exclusively occupy a single trace for isolated evaluation. We use Pantheon [35] as the congestion control evaluation framework to benchmark different schemes in these emulated yet realistic network environments.

Experimental Settings. We use 28 real-world wireless traces in our evaluation [30], [2], [3]. These traces exhibit highly time-varying characteristics, including significant bandwidth fluctuations, dynamic link-quality changes, and substantial diversity across different mobility conditions. They are grouped into three general categories:

- *Stationary scenario*: at home, where the cellular user remains at a fixed location without movement.
- *Pedestrian scenario*: in crowded places, where the user is walking.
- *Vehicular scenario*: in a bus, taxi, or subway, where the user is traveling in public transportation.

Unless otherwise specified, the default propagation delay between the mobile host and the server is set to 20 ms, and the bottleneck buffer size is set to 200 KB, roughly corresponding to 140 packets.

Baselines. We compare DECO against some congestion control schemes, including Cubic [16], BBR [8], Copa [6], Vivace [11], Illinois [20], Indigo [35], LEDBAT [25], PCC [10], Vegas [7], Sprout [30], Verus [37] and Westwood [9]. For DECO, we implement three service configurations: a delay-sensitive mode (DECO-s1), a throughput-oriented mode (DECO-s3), and an intermediate mode that explicitly balances throughput and delay (DECO-s2). The detailed parameter settings are summarized in Table I. For the Sender-based CC selection instantiation, we reuse the same profile parameterization as in the Distributed CC instantiation, but in the experiments below instantiate only the two endpoint service configurations, also named DECO-s1 and DECO-s3.

Evaluation Protocol. Unless otherwise specified, the reported DECO results in the main performance evaluation are obtained under the shared-experience setting, where senders of the same service profile use the corresponding shared model maintained at the Service unit. Moreover, the main results are measured after the model has reached a stable operating condition, so that they reflect the converged/steady-state behavior of DECO rather than the transient exploration phase during startup. The benefit of shared experience for startup adaptation is evaluated separately in Fig. 9a, which isolates the adaptation process instead of the steady-state operating point.

B. Evaluation of Two DECO Instantiations

We next evaluate whether two DECO instantiations can realize the same service abstraction. We first examine Distributed CC over the full trace set to characterize its service operating regions and tail-delay behavior. We then examine whether Sender-based CC selection can preserve the same profile differentiation under the shared DECO framework. As a supplementary study, we report multi-flow fairness convergence for the Distributed CC instantiation.

1) *Distributed CC Instantiation*: We demonstrate the performance of the first instantiation, including the operating point and the tail delay. Fig. 4 plots average one-way delay and link utilization (*i.e.*, goodput/capacity) for all candidates in the three scenarios. The percentile-delay results are reported in Table II.

Fig. 4 shows that DECO can provide different operating points as applications demand. With a delay-sensitive objective, DECO-s1 stays near the low-delay region occupied by Copa and Vegas, while DECO-s3 moves toward the high-utilization region occupied by BBR-, Cubic-, and Westwood-like baselines. An objective pursuing the best operating point lies between these two modes, but mainly serves as an ideal objective rather than the central claim of this paper. What matters is that DECO can map different service objectives to different positions of the delay-link utilization tradeoff curve without changing the service interface.

For throughput-oriented services, DECO-s3 stays close to aggressive throughput baselines in utilization, but avoids

TABLE II: 95th, 99th, and 99.9th percentile one-way delays and link utilization in different types of scenarios.

Scheme	Stationary				Pedestrian				Vehicular			
	P95 (ms)	P99	P999	Util. (%)	P95 (ms)	P99	P999	Util. (%)	P95 (ms)	P99	P999	Util. (%)
DECO-s1	28.23	50.18	195.83	72.82	52.69	131.63	341.61	62.90	42.23	84.21	217.48	66.63
DECO-s2	34.05	59.61	210.10	84.44	67.54	153.87	376.38	76.47	67.32	134.51	379.80	83.61
DECO-s3	57.33	96.43	237.95	92.64	85.65	182.30	491.99	86.25	94.27	188.45	420.04	88.59
BBR	102.06	193.17	325.11	96.60	198.07	372.87	1019.79	95.73	189.01	319.93	597.24	95.29
Cubic	125.65	222.92	374.70	98.67	231.76	417.56	1146.95	98.49	249.85	523.94	837.85	98.37
Copa	27.19	46.59	189.14	73.21	54.37	143.16	398.70	64.51	41.00	84.66	242.16	67.01
Sprout	67.68	137.30	241.71	95.06	110.78	236.16	570.92	88.51	77.71	128.72	250.11	88.89
Westwood	72.38	139.62	269.44	92.35	130.54	258.57	716.99	89.87	123.82	262.24	546.01	89.10
Illinois	118.04	206.49	333.41	98.68	223.22	406.76	1136.59	98.56	225.67	457.42	721.63	98.42
Indigo	21.53	42.69	185.08	35.80	40.29	112.80	338.81	36.29	35.58	72.45	175.50	33.02
LEDBAT	81.71	156.57	272.71	93.46	130.23	237.31	759.25	91.76	110.42	180.78	346.87	89.80
PCC	45.88	149.86	268.54	50.37	150.43	313.95	1028.04	55.97	147.88	345.48	639.77	43.56
Verus	54.06	128.37	241.75	74.61	113.93	263.83	980.12	67.85	99.57	203.47	468.92	66.44
Vegas	27.31	56.58	198.06	66.04	59.17	160.42	404.22	62.29	41.28	82.78	198.29	59.30
Vivace	58.06	176.52	262.91	32.15	182.14	396.76	1025.46	32.69	229.37	504.07	773.57	36.35

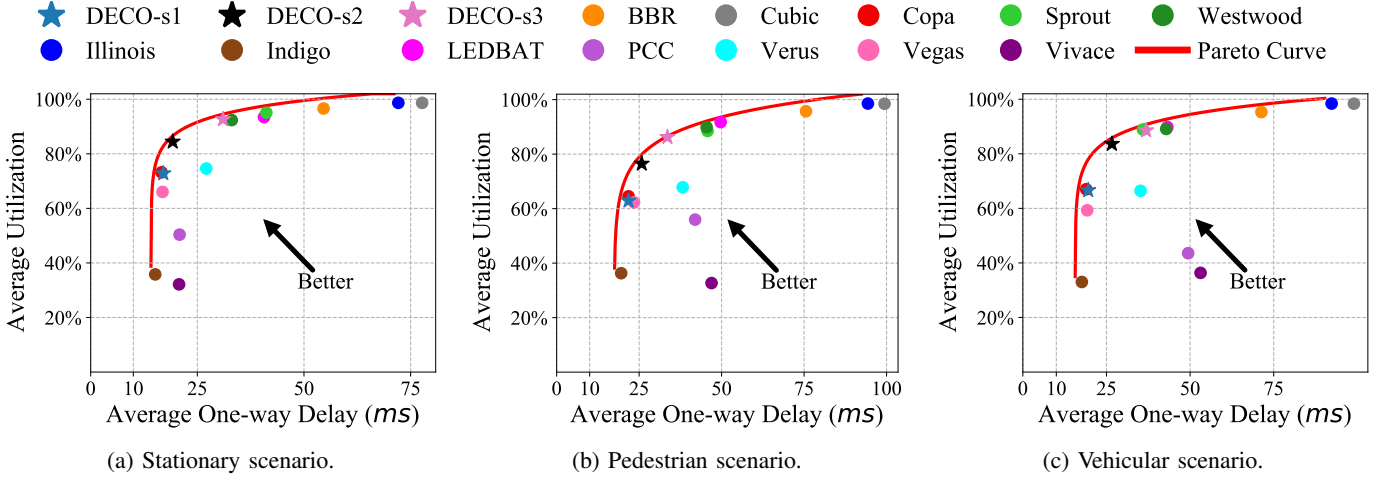


Fig. 4: Average utilization (goodput / capacity) and one-way delay in three types of scenarios.

much of their latency cost. Across the three scenario types, it retains about 90%–96% of BBR’s utilization and about 88%–94% of Cubic’s, while reducing P999 delay by 27%–52% relative to BBR and by 36%–57% relative to Cubic. This shows that DECO can provide a throughput-oriented service without inheriting the full queueing penalty of always-aggressive controllers. For delay-sensitive services, DECO-s1 remains in the Copa/Vegas region and accepts throughput waste. Its link utilization stays within about 3% of Copa across all three scenarios and exceeds Vegas by up to 12%. In pedestrian traces, it also reduces P999 delay by about 14%–15% relative to both Copa and Vegas while keeping utilization comparable. The balanced mode, DECO-s2, remains between these two service extremes and mainly illustrates that the same framework can expose additional operating points when needed. Together, these results show that DECO preserves service differentiation even as wireless conditions become more dynamic.

In addition, DECO-s1 remains close to the Copa/Vegas class in high-percentile delay and, in the more dynamic pedestrian setting, further reduces tail delay by about 15%. For throughput-oriented services, DECO-s3 shows better tail delay control than aggressive high-utilization baselines while maintaining comparable utilization. For example, in pedestrian traces, it achieves 86.25% utilization, versus 95.73%

for BBR and 98.49% for Cubic, while reducing P99 delay from 372.87/417.56 ms to 182.30 ms and P999 delay from 1019.79/1146.95 ms to 491.99 ms. This result is particularly relevant for wireless uplink traffic, where service quality is often more sensitive to queue buildup and tail inflation than to average performance alone.

These results reinforce the main point of DECO that different services should not be forced to share one fixed operating point. Instead, the framework can provide throughput-oriented and delay-sensitive behaviors under a common service abstraction, each aligned with the baseline family that best reflects that service objective.

2) *Sender-based CC Selection Instantiation:* We next evaluate the second instantiation introduced in Section IV-B on the stationary scenario and vehicular scenario. Under the same DECO interface, the Service unit selects controllers from {Vegas, BBR, Cubic} according to the service profile and telemetry summaries, and the Terminal executes the selected controller locally. In this implementation, controller selection is refreshed every $K = 125$ monitoring periods (10 s), and switching is triggered only when the relative score improvement exceeds $\beta_{sw} = 0.1$.

Fig. 5 shows that the second instantiation preserves clear profile differentiation. On both traces, DECO-s1 stays very close to Vegas, while DECO-s3 stays close to BBR/Cubic.

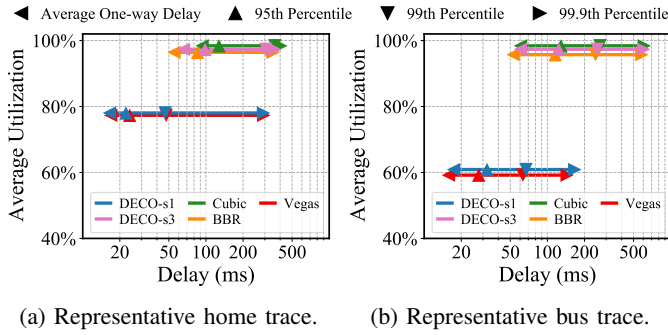


Fig. 5: Performance of the Sender-based CC selection instantiation on two representative traces.

For example, on the home trace, DECO-s1 achieves 78.03% utilization with 16.46 ms average one-way delay, which is nearly identical to Vegas (77.34%, 16.85 ms), whereas DECO-s3 reaches 97.33% utilization, close to BBR (96.43%) and Cubic (98.40%). On the bus trace, DECO-s1 again remains close to Vegas, while DECO-s3 overlaps the throughput-oriented region of BBR and Cubic.

The percentile-delay markers show a similar pattern. DECO-s1 almost matches Vegas across P95, P99, and P999 delays on both traces, while DECO-s3 remains close to BBR/Cubic in the high-utilization regime. Therefore, although this instantiation does not perform fine-grained learned control, it can still realize profile-aligned operating regions under the same DECO interface.

Overall, this result shows that the proposed DECO is not tied to a single solver; it is potential to provide on-demand policy support for diverse senders.

3) *Multi-flow Fairness Convergence*: We evaluate multi-flow fairness convergence. The setup uses a fixed link capacity of 48 Mbps, a round-trip time of 20 ms between the mobile host and the server, a bottleneck buffer of 200 KB, and no random packet loss. This is intended as a supplementary study rather than a primary wireless result, since in many wireless systems fairness is often partly provided by lower-layer scheduling. Here, we use DECO-s2 to illustrate the case where fairness also needs to be maintained at the congestion-control layer. In this setting, DECO can support fairness convergence while tracking an appropriate operating point under flow competition.

Fig. 6 compares multi-flow fairness convergence under six congestion-control schemes. All schemes eventually move toward relatively fair sharing after multiple flows join, but their transient behaviors differ substantially. DECO reacts more smoothly to each new flow arrival and quickly redistributes bandwidth without allowing earlier flows to dominate for an extended period. In contrast, Cubic, BBR, Westwood, Vegas, and Illinois exhibit larger rate oscillations or slower convergence after new flows enter the competition.

Overall, these results show that DECO achieves the best fairness convergence behavior among the evaluated schemes by combining fast convergence with stable rate adjustment. This complements the earlier performance results and suggests that DECO can maintain balanced multi-flow behavior while

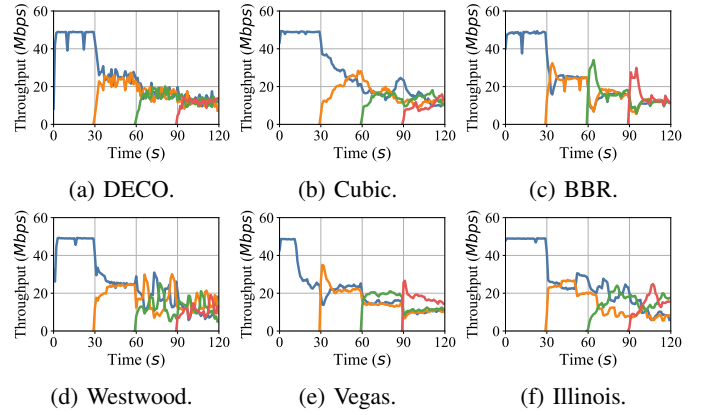


Fig. 6: Multi-flow fairness convergence.

operating near a suitable working point.

C. Framework-Oriented Evaluation

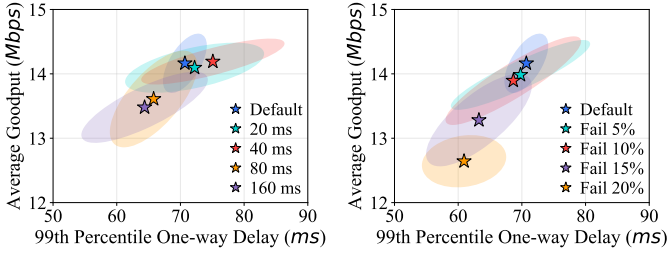
We next evaluate whether DECO is practical as a deployable end-edge framework beyond single-instantiation operating points. We first examine robustness when the control service becomes slower or less reliable, and then quantify deployability in terms of Terminal-side overhead, edge-side scalability, control-plane communication cost, and experience reuse. Finally, we study whether DECO still preserves profile-aligned behavior during coexistence with legacy congestion-control algorithms.

1) *Robustness to Control-Service Degradation*: We next evaluate the robustness of DECO when the control service becomes slower or partially unavailable. In this experiment, we inject additional service-response delay or a non-zero response failure probability. When a response fails, the sender does not receive a fresh policy and continues with its most recent valid policy. The default configuration is full response, with a 10 ms one-way control-plane propagation latency. In this experiment, we used DECO-s2.

Longer Service-Response Delay. Fig. 7a shows the impact of increasing the service-response delay. Overall, the operating point shifts only mildly as the delay increases. Average goodput decreases slightly, while the 99th percentile one-way delay also decreases slightly. This trend indicates that slower policy refresh mainly makes DECO less aggressive, rather than destabilizing the control loop.

Response Failures. Fig. 7b shows a similar trend when the control service becomes less reliable. As the response failure probability increases, the operating point gradually moves toward lower goodput and lower 99th percentile delay. This again indicates a more conservative behavior. The degradation is smooth, which suggests that intermittent service failures mainly reduce adaptation timeliness but do not break the basic control behavior.

In conclusion, both larger control-plane delay and lower response availability make DECO slightly less aggressive. Even under increased control-plane RTT or non-negligible response loss, the impact remains small, and the system continues to operate near an acceptable region.



(a) Impact of additional service-response delay. (b) Impact of service-response failure probability.

Fig. 7: Average goodput versus 99th percentile one-way delay under degraded control-service conditions.

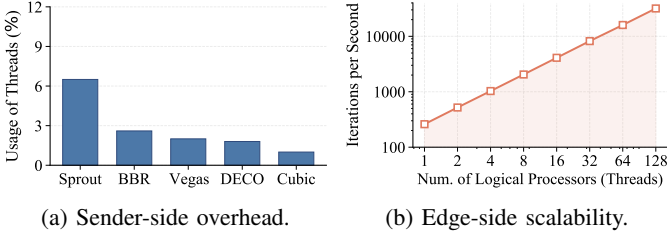


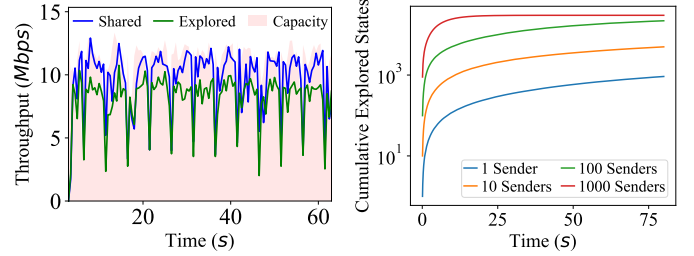
Fig. 8: Runtime overhead of DECO.

2) *Overhead, Scalability, and Experience Reuse*: We next evaluate several practical aspects of DECO beyond end-to-end transmission performance, including runtime overhead, edge-side scalability, and the benefit of experience reuse. Since DECO follows an end-edge collaborative design, its runtime cost is naturally divided between the Terminal-side and edge-side components. The Terminal mainly performs lightweight local measurement and policy execution, while the computationally intensive tasks, such as experience aggregation and policy update, are offloaded to the edge. In our testbed, Mahimahi-based sender emulation is distributed across multiple hosts to generate concurrent trace-driven traffic, whereas the scalability results below focus specifically on the shared-model training path at the Service unit.

Low Terminal-Side Overhead and Scalable Edge Execution. Fig. 8a shows that the runtime overhead on the Terminal side remains very low. Compared with conventional congestion control schemes, DECO introduces only a small additional Terminal-side cost, which is consistent with its lightweight Terminal-side design. Fig. 8b measures the throughput of edge-side training rather than the host-level scaling of the Mahimahi environment or the exact number of served Terminals. In our implementation, Terminals of the same service type share a global model, while multiple worker threads train this model asynchronously within a single process. As the number of logical processors increases, the aggregate training throughput grows almost linearly. With 128 threads, the edge server can sustain nearly 32,000 aggregate training iterations per second. Since many Terminals share the same model and only periodically contribute experience, this raw training throughput suggests that the Service unit can support substantially more participating Terminals than the number of concurrent training workers. These results indicate that Service-unit training overhead remains modest even under

TABLE III: Estimated communication overhead of DECO.

Direction	Message type	Payload size	Interval	Overhead
Uplink	Telemetry report	100 bytes	80 ms	10.0 kbps
Uplink	Telemetry report	100 bytes	160 ms	5.0 kbps
Uplink	Telemetry report	100 bytes	320 ms	2.5 kbps
Downlink	Rule-based policy	100 bytes	5 s	0.16 kbps
Downlink	Rule-based policy	100 bytes	10 s	0.08 kbps
Downlink	Rule-based policy	100 bytes	20 s	0.04 kbps
Downlink	DRL model policy	100 KB	5 s	160.0 kbps
Downlink	DRL model policy	100 KB	10 s	80.0 kbps
Downlink	DRL model policy	100 KB	20 s	40.0 kbps



(a) Exploration speed with different numbers of senders. (b) Shared experience vs. individual exploration.

Fig. 9: Benefits of shared experience in DECO.

high concurrency.

Low Communication Overhead. In addition to the modest computation cost, DECO incurs only limited control-plane communication overhead. We use a conservative estimate with a 100-byte telemetry report and a 100-KB DRL model policy object. Under a representative setting, each Terminal uploads one telemetry report every 80 ms, corresponding to about 10.0 kbps of uplink overhead, and refreshes its local model from the edge every 20 s, corresponding to about 40.0 kbps of downlink overhead. Table III summarizes the estimated uplink and downlink overhead under different reporting and refresh intervals. Overall, the communication cost remains small relative to the service traffic. The performance impact of degraded control-plane conditions is evaluated separately in Section V-C1.

Shared Experience Accumulation. Fig. 9b shows that as the number of senders increases, the cumulative explored-state space grows much faster, because more concurrent interactions generate more reusable experience. This growth is sublinear in the large-sender regime, indicating diminishing marginal benefit rather than unbounded gain.

Faster Startup Adaptation. Fig. 9a then shows how this shared experience improves adaptation behavior. Without cold-start support or shared experience, a Terminal must rely only on its own local exploration history, so its local model remains under-converged for a longer period and therefore behaves more conservatively, leading to lower throughput during the early stage. By contrast, with shared experience, the Terminal starts from a better-informed model and can approach the available capacity more quickly, thereby shortening the conservative exploration phase and avoiding prolonged weak operating points.

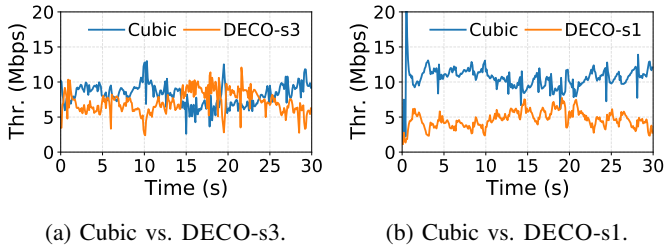


Fig. 10: Flow competition between DECO and Cubic.

3) *Heterogeneous Flow Competition*: Finally, we evaluate heterogeneous flow competition when DECO coexists with flows using a conventional congestion control algorithm. Fig. 10 shows the throughput evolution of two competing flows under different DECO service profiles, where one flow uses Cubic and the other uses DECO.

Throughput-Oriented Competition. As shown in Fig. 10a, when competing against Cubic, DECO-s3 maintains a throughput level in the same general range and exhibits active bandwidth competition throughout the run. Although Cubic remains slightly more aggressive in some periods, DECO-s3 is able to sustain continuous competition without being persistently suppressed, indicating that the throughput-oriented profile can effectively contend for bandwidth against a conventional loss-based baseline.

Delay-Sensitive Competition. Fig. 10b shows a different behavior under the delay-sensitive profile. In this case, DECO-s1 consistently operates at a lower throughput level than Cubic, reflecting its more conservative service objective. Rather than aggressively matching Cubic’s sending rate, DECO-s1 intentionally yields part of the bandwidth in exchange for a milder operating point. This result is consistent with the design goal of service differentiation: different DECO profiles exhibit different competitive behaviors under the same heterogeneous environment.

Overall, these results show that DECO can preserve profile-dependent operating behavior even under cross-algorithm competition. The throughput-oriented mode remains competitive against Cubic, while the delay-sensitive mode maintains a more conservative operating point, further demonstrating that DECO does not enforce a fixed congestion-control behavior across services.

VI. RELATED WORK

Classical and Scenario-Specific Congestion Control.

Prior work has developed a wide range of congestion control algorithms, including classical loss-based schemes such as TCP Tahoe/Reno, NewReno, BIC, and Cubic [17], [13], [34], [16], delay or model-aware designs such as Vegas, Compound, Illinois, Copa, and BBR [7], [27], [20], [6], [8], as well as scenario-specific schemes for datacenter and wireless/cellular environments, such as DCTCP, DCQCN, Sprout, Verus, and C2TCP [5], [14], [41], [30], [37], [1]. Although these methods differ in signals and control rules, they largely remain endpoint-centric, where control behavior is mainly determined by sender-side measurements, fixed rules, or relatively static parameter settings.

Learning-Based Congestion Control. Learning-based approaches seek to improve adaptability under complex and time-varying network conditions. Remy pioneered data-driven congestion control design [29]. PCC Allegro and PCC Vivace optimize control through online utility-driven adaptation [10], [11], while KerDqn, Indigo, Aurora, Orca, and DeepCC further explore q-learning or deep reinforcement learning for more flexible congestion control [33], [35], [18], [2], [3]. While these methods improve adaptability, they still mainly focus on endpoint-side controller design and often face challenges in generalization, online updating, and overhead.

QoS-Aware and Service-Oriented Congestion Control. Recent studies have explored adaptive mechanisms to better align congestion control with time-varying service or QoS objectives [40], [36], [38], [21], [26]. These efforts mainly focus on dynamic controller selection or parameter tuning during runtime. In contrast, our work formulates wireless uplink congestion control as a reusable end-edge service capability with service profiles, shared experience reuse, bootstrap support, and safe fallback, rather than controller-local adaptation logic.

VII. CONCLUSION

We presented DECO, a service-oriented end-edge collaborative congestion-control framework for mobile uplink traffic. Rather than seeking one best controller, DECO reformulates congestion control as expressing, aligning with, and tracking service-dependent QoS regions under mobility-induced dynamics. By decoupling objective expression from local execution through service profiles and an independent control plane, DECO enables edge-assisted policy generation, shared experience reuse, bootstrap initialization, and graceful degradation while keeping Terminal execution lightweight.

We primarily instantiate this framework through distributed CC, where a DRL-based policy engine at the Service unit learns profile-aware control from shared edge-side experience and the Terminal performs lightweight local execution. We also examine sender-based CC selection as a secondary realization, mainly to show that the same service abstraction can support terminals that execute conventional sender-side controllers. Evaluations over 28 real-world mobile wireless traces demonstrate that DECO provides differentiated service modes: the throughput-oriented mode preserves most of BBR’s utilization while substantially reducing tail delay, and the delay-sensitive mode operates near the Copa/Vegas low-delay region with lower tail delay on pedestrian traces. Additional experiments show that DECO maintains favorable multi-flow fairness convergence, remains robust under degraded control-service conditions, and scales efficiently at the edge. These results suggest that congestion control for mobile uplinks can evolve from isolated endpoint logic into a reusable networked service capability. Future work will extend DECO toward richer service intents, broader wireless deployment environments, and more diverse terminal-side control capabilities.

REFERENCES

- [1] Soheil Abbasloo, Yang Xu, and H Jonathan Chao. C2tcp: A flexible cellular tcp to meet stringent delay requirements. *IEEE Journal on Selected Areas in Communications*, 37(4):918–932, 2019.

- [2] Soheil Abbasloo, Chen-Yu Yen, and H Jonathan Chao. Classic meets modern: A pragmatic learning-based congestion control for the internet. In *Proceedings of the Annual conference of the ACM Special Interest Group on Data Communication on the applications, technologies, architectures, and protocols for computer communication*, pages 632–647, 2020.
- [3] Soheil Abbasloo, Chen-Yu Yen, and H Jonathan Chao. Wanna make your tcp scheme great for cellular networks? let machines do it for you! *IEEE Journal on Selected Areas in Communications*, 39(1):265–279, 2020.
- [4] Neil Agarwal, Rui Pan, Francis Y Yan, and Ravi Netravali. Mowgli: Passively learned rate control for {Real-Time} video. In *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 25)*, pages 579–594, 2025.
- [5] Mohammad Alizadeh, Albert Greenberg, et al. Data center tcp (dctcp). In *Proceedings of the ACM SIGCOMM 2010 Conference*, pages 63–74, 2010.
- [6] Venkat Arun and Hari Balakrishnan. Copa: Practical {Delay-Based} congestion control for the internet. In *15th USENIX Symposium on Networked Systems Design and Implementation (NSDI 18)*, pages 329–342, 2018.
- [7] Lawrence S Brakmo, Sean W O’malley, and Larry L Peterson. Tcp vegas: New techniques for congestion detection and avoidance. In *Proceedings of the conference on Communications architectures, protocols and applications*, pages 24–35, 1994.
- [8] Neal Cardwell, Yuchung Cheng, C Stephen Gunn, Soheil Hassas Yeganeh, and Van Jacobson. Bbr: Congestion-based congestion control: Measuring bottleneck bandwidth and round-trip propagation time. *Queue*, 14(5):20–53, 2016.
- [9] Claudio Casetti, Mario Gerla, et al. Tcp westwood: end-to-end congestion control for wired/wireless networks. *Wireless Networks*, 8(5):467–479, 2002.
- [10] Mo Dong, Qingxi Li, et al. {PCC}: Re-architecting congestion control for consistent high performance. In *12th USENIX Symposium on Networked Systems Design and Implementation (NSDI 15)*, pages 395–408, 2015.
- [11] Mo Dong, Tong Meng, et al. {PCC} vivace: {Online-Learning} congestion control. In *15th USENIX Symposium on Networked Systems Design and Implementation (NSDI 18)*, pages 343–356, 2018.
- [12] Xinle Du, Jie Li, et al. Revisiting congestion control for wifi networks. In *Proceedings of the 8th Asia-Pacific Workshop on Networking*, pages 88–94, 2024.
- [13] Sally Floyd, Tom Henderson, and Andrei Gurtov. The newreno modification to tcp’s fast recovery algorithm. Technical report, 2004.
- [14] Sally Floyd, Dr. K. K. Ramakrishnan, and David L. Black. The Addition of Explicit Congestion Notification (ECN) to IP. RFC 3168, September 2001.
- [15] Prateesh Goyal, Anup Agarwal, et al. {ABC}: A simple explicit congestion controller for wireless networks. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*, pages 353–372, 2020.
- [16] Sangtae Ha, Injong Rhee, and Lisong Xu. Cubic: a new tcp-friendly high-speed tcp variant. *ACM SIGOPS operating systems review*, 42(5):64–74, 2008.
- [17] Van Jacobson. Congestion avoidance and control. *ACM SIGCOMM computer communication review*, 18(4):314–329, 1988.
- [18] Nathan Jay, Noga Rotman, et al. A deep reinforcement learning perspective on internet congestion control. In *International Conference on Machine Learning*, pages 3050–3059. PMLR, 2019.
- [19] Yang Li, Jiaxing Qiu, et al. Dissecting and streamlining the interactive loop of mobile cloud gaming. In *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 25)*, pages 595–611, 2025.
- [20] Shao Liu, Tamer Başar, and Ravi Srikant. Tcp-illinois: A loss and delay-based congestion control algorithm for high-speed networks. In *Proceedings of the 1st international conference on Performance evaluation methodologies and tools*, pages 55–es, 2006.
- [21] Yiqing Ma, Han Tian, et al. Multi-objective congestion control. In *Proceedings of the Seventeenth European Conference on Computer Systems*, pages 218–235, 2022.
- [22] Zili Meng, Yaning Guo, et al. Achieving consistent low latency for wireless real-time communications with the shortest control loop. In *Proceedings of the ACM SIGCOMM 2022 Conference*, pages 193–206, 2022.
- [23] Ravi Netravali, Anirudh Sivaraman, Somak Das, Ameesh Goyal, Keith Winstein, James Mickens, and Hari Balakrishnan. Mahimahi: accurate {Record-and-Replay} for {HTTP}. In *2015 USENIX Annual Technical Conference (USENIX ATC 15)*, pages 417–429, 2015.
- [24] Yunzhe Ni, Zhilong Zheng, et al. Cellfusion: Multipath vehicle-to-cloud video streaming with network coding in the wild. In *Proceedings of the ACM SIGCOMM 2023 Conference*, pages 668–683, 2023.
- [25] Dario Rossi, Claudio Testa, Silvio Valenti, and Luca Muscariello. Ledbat: the new bittorrent congestion control protocol. In *2010 Proceedings of 19th International Conference on Computer Communications and Networks*, pages 1–6. IEEE, 2010.
- [26] Yixin Shen, Ruihua Chen, et al. Mortise: Auto-tuning congestion control to optimize QoE via Network-Aware parameter optimization. In *23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI 26)*, pages 2303–2321, Renton, WA, May 2026. USENIX Association.
- [27] Kun Tan, Jingmin Song, Qian Zhang, and Murad Sridharan. A compound tcp approach for high-speed and long distance networks. In *Proceedings-IEEE INFOCOM*, 2006.
- [28] Han Tian, Wenbo Li, et al. Learn-to-probe: Achieving signal distinguishability in learning-based congestion control. In *Proceedings of the 21st European Conference on Computer Systems*, pages 108–125, 2026.
- [29] Keith Winstein and Hari Balakrishnan. Tcp ex machina: Computer-generated congestion control. *ACM SIGCOMM Computer Communication Review*, 43(4):123–134, 2013.
- [30] Keith Winstein, Anirudh Sivaraman, and Hari Balakrishnan. Stochastic forecasts achieve high throughput and low delay over cellular networks. In *10th USENIX Symposium on Networked Systems Design and Implementation (NSDI 13)*, pages 459–471, 2013.
- [31] Yaxiong Xie, Fan Yi, and Kyle Jamieson. Pbe-cc: Congestion control via endpoint-centric, physical-layer bandwidth measurements. In *Proceedings of the Annual conference of the ACM Special Interest Group on Data Communication on the applications, technologies, architectures, and protocols for computer communication*, pages 451–464, 2020.
- [32] Heng Xu, Letian Li, et al. Congestion control as a service: Towards low latency mobile uploading. In *2024 IEEE/ACM 32nd International Symposium on Quality of Service (IWQoS)*, pages 1–10, 2024.
- [33] Heng Xu, Liang Wang, and Fei Song. Kerdqn: Deep reinforcement learning enhanced congestion control in kernel. In *ICC 2024-IEEE International Conference on Communications*, pages 479–484. IEEE, 2024.
- [34] Lisong Xu, Khaled Harfoush, and Injong Rhee. Binary increase congestion control (bic) for fast long-distance networks. In *IEEE INFOCOM 2004*, volume 4, pages 2514–2524. IEEE, 2004.
- [35] Francis Y Yan, Jestin Ma, et al. Pantheon: the training ground for internet congestion-control research. In *2018 USENIX Annual Technical Conference (USENIX ATC 18)*, pages 731–743, 2018.
- [36] Furong Yang, Zhenyu Li, et al. Disco: A framework for dynamic selection of multipath congestion control algorithms. In *2023 IEEE 31st International Conference on Network Protocols (ICNP)*, pages 1–12. IEEE, 2023.
- [37] Yasir Zaki, Thomas Pötsch, Jay Chen, Lakshminarayanan Subramanian, and Carmelita Görg. Adaptive congestion control for unpredictable cellular networks. In *Proceedings of the 2015 ACM Conference on Special Interest Group on Data Communication*, pages 509–522, 2015.
- [38] Jia Zhang, Yixuan Zhang, et al. Bridging the gap between {QoE} and {QoS} in congestion control: A large-scale mobile web service perspective. In *2023 USENIX Annual Technical Conference (USENIX ATC 23)*, pages 553–569, 2023.
- [39] Anfu Zhou, Huanhuan Zhang, et al. Learning to coordinate video codec with transport protocol for mobile video telephony. In *The 25th Annual International Conference on Mobile Computing and Networking*, pages 1–16, 2019.
- [40] Jianer Zhou, Xinyi Qiu, et al. Antelope: A framework for dynamic selection of congestion control algorithms. In *2021 IEEE 29th International Conference on Network Protocols (ICNP)*, pages 1–11. IEEE, 2021.
- [41] Yibo Zhu, Haggai Eran, et al. Congestion control for large-scale rdma deployments. *ACM SIGCOMM Computer Communication Review*, 45(4):523–536, 2015.